

안녕하세요,
사용자의 작업 흐름을 개선하는 개발자 박세현입니다.



phone

010-6210-4485

email

pigkill40@naver.com

github

<https://github.com/pakxe>

SUMMARY

로봇 상태 시각화 및 제어 대시보드, 실시간 협업 마인드맵, 지도 기반 경로 편집 서비스를 개발했습니다.
지도 및 공간 데이터와 실시간 협업에서 발생하는 복잡한 인터랙션을 다루고, 브라우저 성능을 최적화했습니다.
실제 사용자의 작업을 관찰해 좌표를 직접 입력하던 방식을 GUI 기반 편집으로 전환하고, 반복 작업의 불편을 개선했습니다

CAREER

NAVER LABS

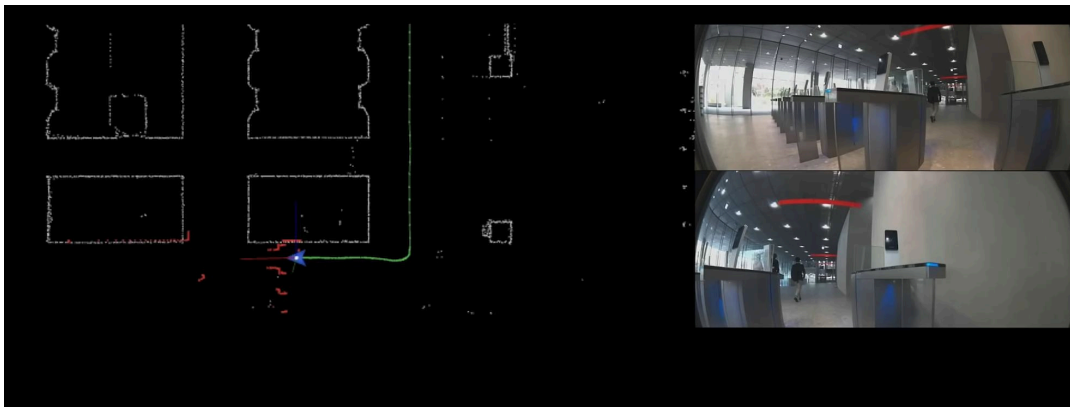
Full-stack Intern · Robot Vision & Learning Team

2025.03 - 2025.09

- 로봇의 다층 이동 테스트를 지원하는 웹 대시보드 및 서버 개발. 기존 시각화 프로그램인 rviz에서 관찰하기 불편했던 다층 이동 상황을 지원하기 위해 개발했으며, 다층 이동을 포함한 로봇 이동 테스트에 활용
- 로봇 토픽을 구독하고 데이터를 websocket으로 전달하는 서버와, 로봇 위치, 상태, 센서 데이터, 카메라 영상을 시각화하는 클라이언트 구현. 웨이포인트 수정, 초기 위치 설정 등의 기능 제공
- 주간 회의에서 연구자의 요구사항을 확인하고, Figma 프로토타입 제작·검토를 거쳐 기능 클라이언트 및 서버 구현
- 로봇 - fastAPI 서버 - react 클라이언트로 구성. 서버가 로봇 토픽을 구독해 수신한 데이터를 websocket으로 클라이언트에 전달, 클라이언트에서 입력한 이동 경로와 초기 위치 설정 등은 서버를 통해 로봇에 전달하는 양방향 통신 구조 구현

react, three.js, react-three-fiber, leaflet, zustand, emotion, storybook

일부 작업물



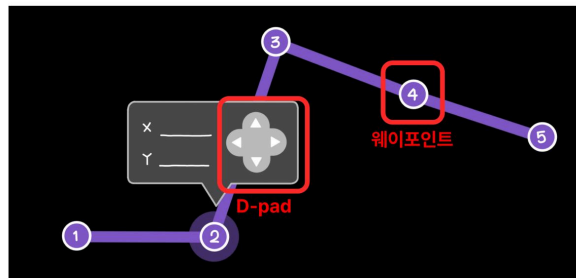
<https://youtu.be/WG3WV0l0ln8?si=3wjsZwF2x1-ZWxNK&t=11>

구조물 식별과 사용자 입력 지연을 고려한 대규모 포인트 클라우드 렌더링 최적화

- 문제** 2D 지도에서 수십만 개의 포인트 클라우드를 반복적으로 탐색하고 렌더링하면서 로봇 tracking 중 화면이 끊기고 브라우저 조작이 어려워졌습니다.
- 선택** 처음에는 일정 간격으로 포인트를 추출했으나, 구조물 식별이 어려워 voxelization으로 변경했습니다. 이후 전체 순회 비용을 줄이기 위해 quad tree를 사용하여 필터링하였고, viewport 내부만 query해 처리량을 줄였습니다. quad tree 구축, viewport 검색은 web worker에서 수행해, 해당 연산으로 인한 메인 스레드의 blocking을 줄이고자 했습니다.
- 결과** 로컬 chrome performance에서 최적화 후에 지속적인 heap 증가가 관찰되지 않았고 브라우저 중단 현상이 재현되지 않았습니다. 최종 구현물은 실제 로봇 tracking에 사용되었습니다.

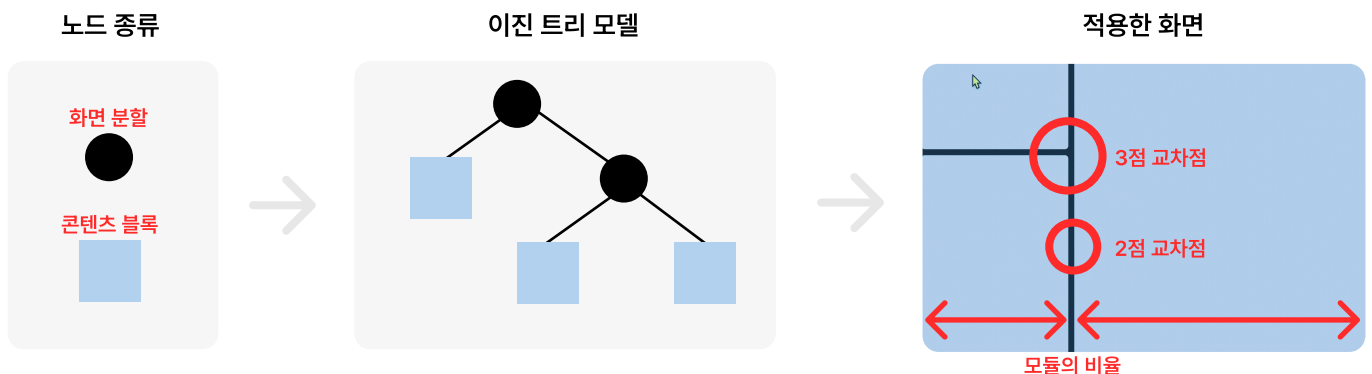
좌표를 직접 입력하는 CLI 방식을 지도 기반 GUI로 전환

- 문제** 새로운 이동 경로나 시작 위치를 설정하려면 터미널에서 좌표 파일을 수정하거나 미리 작성된 셀 스크립트를 실행해야 했습니다.
- 선택** 연구자가 좌표를 몰라도 지도에서 정확한 위치를 선택할 수 있도록 지도 클릭 기반 웨이포인트 편집과 D-pad를 제공했습니다. 테스트 로봇이 지도에 정의된 노드를 따라 이동하는 경우가 많다는 점을 고려해, 지도 그래프에 Dijkstra 경로 탐색을 적용했습니다. 시작, 도착 노드만 선택하면 경로상의 웨이포인트가 자동 생성되도록 구현해 중간 노드를 하나씩 선택하는 작업을 줄였습니다. 시작 위치 설정은 층 선택 → 위치 클릭 → 방향 선택 순서로 구성했습니다.
- 결과** 경로 입력과 시작 위치 설정이 대시보드 안에서 완료되어 해당 작업을 위해 터미널을 켜거나 텍스트 수정을 하지 않게 되었습니다. 연구자들이 실제 로봇 실험과 엘리베이터 다층 이동 데모에 사용하였고, 자동 경로 생성이 반복 클릭보다 편리하다는 피드백을 받았습니다.



연구자별 화면 구성과 배치를 지원하는 레이아웃

- 문제** 연구자마다 확인하려는 모듈과 화면 배치가 달랐고, 프로그램을 실행할 때마다 유사한 배치를 구성하는 모습을 확인했습니다.
- 선택** 화면 분할과 콘텐츠 블록을 표현하는 이진 트리 모델을 typescript로 구현했습니다. 각 모듈의 비율과 모듈 위치 조정을 지원하고, 모듈이 교차하는 2, 3점 교차점을 조절하여 모듈의 비율을 조절할 수 있는 동작을 추가했습니다. 해당 트리를 json으로 직렬화해 DB에 저장하고, 대시보드에서 불러올 수 있도록 preset 기능도 추가했습니다.
- 결과** 연구자들이 대시보드를 실행할 때마다 preset을 불러와 기존 화면 구성을 재사용했습니다.



PROJECT

EPISODE 취업 준비생의 경험 정리를 돕는 실시간 협업 마인드맵

Front-end engineer

2026.01 - 2026.02

[github](#)

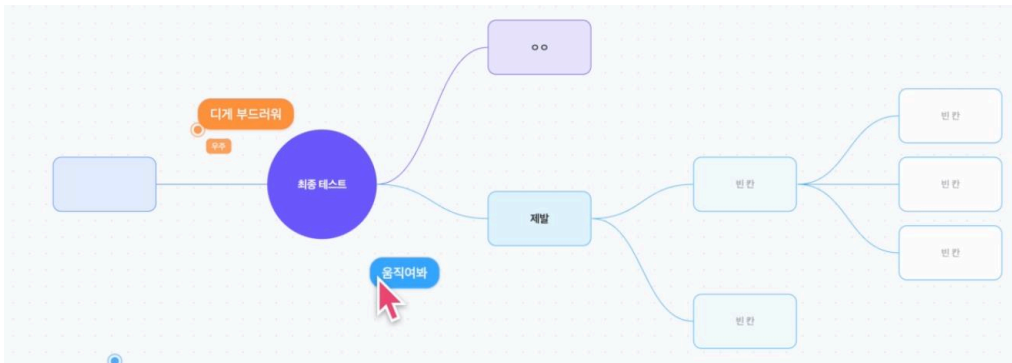
[배포URL](#)

[서비스 사용해보기](#)

- frontend 2명이 기능 목록, 자료구조 및 클래스별 책임을 공동 검토한 뒤 책임 단위로 개발
- 한 달 동안 핵심인 마인드맵과 공동 편집 기능을 구현해 최종 심사에서 High Quality 상 수상

react, typescript, tailwind, tanstack-query, y.js

일부 작업물



관심 상태별 구독 분리로 마인드맵 리렌더링 범위 축소

문제 한 노드의 내용을 변경해도 마인드맵의 모든 노드가 리렌더링됐습니다. 노드 수가 늘어나면 불필요한 리렌더링이 프레임 저하와 입력 지연으로 이어질 수 있다고 판단해, 변경이 필요한 컴포넌트만 리렌더링되도록 개선하고자 했습니다.

선택 각 컴포넌트가 필요한 상태 변경에만 반응하도록, 하나의 snapshot으로 전달하던 상태를 개별 노드 내용, 선택 상태, 사용자 상호작용 등으로 분리했습니다. 또한 memo를 적용해 변경이 불필요한 노드의 리렌더링을 방지했습니다.

결과 react devtools 업데이트 하이라이트로 확인한 결과, 변경되지 않은 노드의 리렌더링이 생략되는 것을 확인했습니다.

공동 편집 중 이동, 삭제 충돌에 대한 조작 제한

문제 여러 사용자가 관련 노드를 동시에 이동하거나 삭제하면 트리 구조가 손상되어 애플리케이션이 중단됐습니다. 공동 편집을 안정적으로 제공하려면, 한 사용자의 조작이 다른 사용자의 편집 대상에 영향을 주는 상황을 제어해야 했습니다.

선택 노드의 조상과 자손 관계를 기준으로 동시 조작을 제한하는 규칙을 설계하고 구현했습니다. 사용자 간 드래그 대상과 잠금 상태를 실시간으로 공유하고, 다른 사용자가 해당 노드와 조상 또는 자손 노드를 이동하거나 삭제하지 못하도록 제한했습니다. 드래그 종료 시에는 잠금을 해제해 다시 편집할 수 있도록 했습니다.

결과 다른 사용자의 편집 대상에 영향을 주는 이동 및 삭제를 UI 단계에서 제한해, 공동 편집 중 트리 구조를 보호했습니다.

네트워크 전환과 탭 복귀에 대응하는 세션 재연결

- 문제** 탭 비활성화나 네트워크 연결 변경 이후 팀 마인드맵 세션이 복구되지 않는 상황이 있었습니다. 사용자가 다른 창으로 이동하거나 네트워크를 바꾼 뒤에도 팀 마인드맵 편집을 수행할 수 있어야 합니다.
- 선택** online, visibilitychange, focus 이벤트와 websocket 연결 상태를 세션 복구를 시작해야하는 시점으로 활용했습니다.
- 결과** 탭 전환과 네트워크 전환 환경에서 세션 재연결 동작을 확인했습니다.

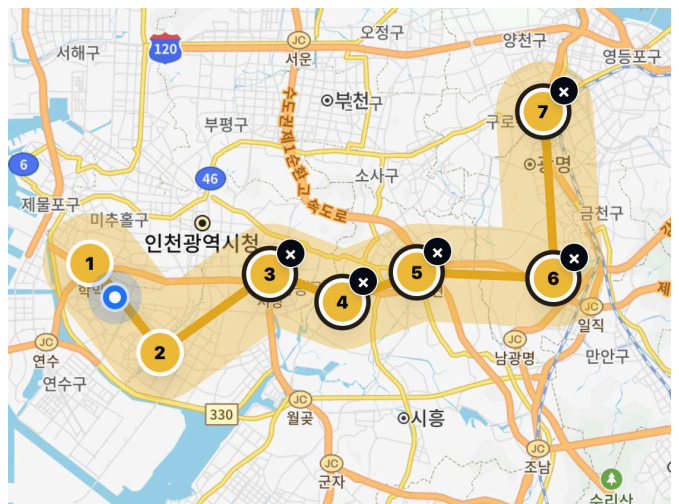
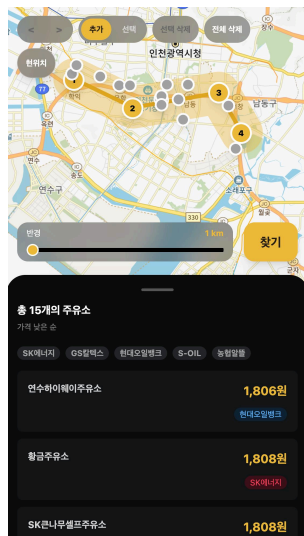
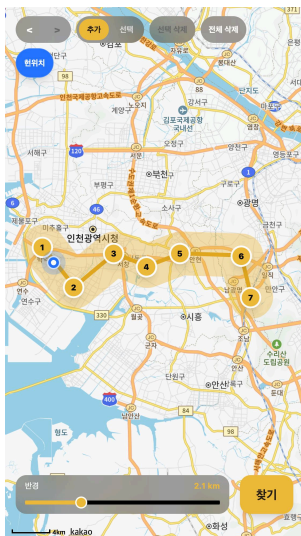
gil-charge 지도에 이동 경로를 그리고 경로 주변 주유소 가격을 검색하는 모바일 우선 웹 애플리케이션

Front-end engineer
2026.08 - 2026.09

[github](#) [배포URL](#) [서비스 사용해보기](#)

react, typescript, y.js, tailwind, tanstack-query,

일부 작업물



오류 해석과 사용자 복구 정책의 책임 분리

- 문제** 네트워크 오류와 백엔드 오류 코드 해석이 hook과 component에 흩어져있어 같은 실패를 화면마다 다르게 처리할 가능성이 있었습니다. 오류 처리 방식이 함께 있어 백엔드 오류 코드의 변경이 화면 코드 수정으로 이어질 수 있었습니다.
- 선택** 변경의 영향을 관련 계층으로 한정하기 위해, 각 계층이 알아야 하는 정보를 기준으로 책임을 네 단계로 분리했습니다.
- http: 네트워크, axios 오류를 공통 오류로 변환해, 상위 계층이 통신 라이브러리의 오류 형식을 몰라도 처리할 수 있도록 했습니다.
 - request: api 응답 검증 및 백엔드 오류 코드 해석을 담당해, 이후 계층에서 서버의 응답 규격을 직접 해석하지 않도록 했습니다
 - hook: 해석된 오류에 따라 입력 수정, 재시도, 안내 등 복구 정책과 표시 방식을 결정했습니다.
 - component: 결정된 정책에 따라 메시지, toast, inline 오류를 표시하고, 오류 코드 해석이나 복구 정책 판단은 맡지 않도록 했습니다.
- 결과** 실제 백엔드 오류 코드 하나가 제거됐을 때, request와 hook의 관련 함수만 수정했습니다. http와 component는 변경하지 않아, 오류 해석과 정책 변경의 영향이 통신 처리와 화면 표현까지 퍼지지 않는 것을 확인했습니다.

최신 검색 결과를 유지하기 위한 요청 취소와 응답 무시

- 문제** 요청 A 이후 시작한 요청 B가 먼저 완료되면, 늦게 도착한 A의 응답이 최신 검색 결과나 오류 상태를 덮어쓸 수 있었습니다.
- 선택** 호출 화면마다 방어 로직을 작성하지 않아도 최신 요청만 상태를 갱신하도록, 검색 hook에 요청 수명과 응답 반영 여부를 관리하는 책임을 모았습니다. hook에서 AbortController를 관리해 새 요청 시작 시 이전 요청을 취소하고, unmount 시에도 진행 중인 요청을 취소했습니다.
- 결과** deferred promise로 요청 완료 순서를 제어하는 테스트를 작성했습니다. B 시작 시 A가 취소되고, B 완료 후 A가 늦게 성공하더라도 B의 검색 결과가 유지되는 것을 검증했습니다.

편집 기능 확장 시 유지보수 부담을 줄이는 스냅샷 기반 히스토리 관리

- 문제** 웨이포인트 편집에 올가미를 이용한 다중 선택, 일괄 이동, 삭제 등 동작을 추가하면서, 각 작업의 실행 취소를 위한 히스토리도 함께 관리해야 했습니다. 새로운 동작을 추가할 때마다 히스토리 처리 로직을 별도로 구현하는 부담을 줄이고자 했습니다.
- 선택** 히스토리 관리 방식으로 action, diff, snapshot 방식을 비교했습니다. 관리하는 웨이포인트 수가 많지 않아 전체 상태를 저장해도 메모리 부담이 크지 않다고 판단해 Snapshot 방식을 선택했습니다. 또한 snapshot은 작업 전 상태를 그대로 저장한 뒤 필요할 때 복원하는 방식이므로, 각 동작에 대한 변경 내역이나 역연산을 별도로 정의할 필요가 없어 새로운 동작 추가에 부담이 적습니다.
- 결과** 일괄 이동, 삭제처럼 여러 웨이포인트의 좌표와 구성이 한 번에 바뀌는 작업을 하나의 snapshot으로 기록해, 간단하게 undo/redo를 사용할 수 있게 되었습니다.

공유할 검색 조건은 URL에, 작성 중인 경로는 sessionStorage에 분리 저장

- 문제** 검색에 사용한 경로와 조건을 다른 사람과 공유하거나, 새로고침 후 작성 중이던 경로를 이어서 편집하기 어려웠습니다.
- 선택** 다른 사람과 공유해야 하는 확정된 경로, 반경, 브랜드 필터는 URL에 저장했습니다. 작성 중 상태인 경로와 반경은 당사자만 필요하고 공유될 필요가 없기 때문에 sessionStorage에 저장했습니다. 이를 통해 공유가 필요한 정보와 현재 탭에서만 이어갈 작업을 구분했습니다.
- 결과** 검색 결과 URL을 다른 사람이나 다른 기기에서 열어도 동일한 경로·반경·브랜드 필터를 재현할 수 있었습니다. 또한 새로고침 후에도 작성 중이던 경로와 반경을 복원해 작업을 이어갈 수 있었습니다. 잘못된 URL이나 저장 데이터로 인해 문제가 생기지 않도록 좌표, 개수, 반경 정규화 동작을 단위 테스트로 검증했습니다.

EDUCATION

인하대학교 공과대학 2020.03 - 2026.08	정보통신공학과 학사 졸업	평점: 4.03/4.50 (전공 평점: 4.12/4.50)
현대자동차 소프티어 2026.01 - 2026.02	프론트엔드 과정	
우아한테크코스 6기 2024.02 - 2024.11	프론트엔드 과정	
멋쟁이사자처럼 2022.03 - 2023.12	(10기) 프론트엔드 부원 (11기) 프론트엔드 팀장, 운영진	

AWARDS

[High Quality 상] 현대자동차그룹 소프티어 최종 프로젝트 심사	2026.02.27
[대상] 2023 INHA SW NET-Zero 공동해커톤	2023.07.15
[은상] 멋쟁이사자처럼 10주년 기념 해커톤	2022.08.20